Assignment-3 Report

Handwritten Devanagari Digit Recognition App (CNN + Streamlit)

Team: SMALLD

Members: Siddharth Mago (2023101088), Soham Chatterjee (2023101092)

Topic ID: T3.1 Devanagari Digit Recogniser

Title: Handwritten Devanagari Digit Recognition using a Small CNN

Github Repo: https://github.com/SiddharthMago/SMAI-A3

Streamlit App Deployment: https://devanagari-digit-recognizer.streamlit.app/

1. Introduction

This project implements the T3.1 assignment variant: Handwritten Devanagari Digit Recognition. The goal is to build an ML web application that accepts a handwritten Devanagari digit through either a drawable canvas or image upload, and predicts the corresponding Unicode digit class. This is useful for Indic educational tools, form digitisation, accessibility interfaces and document processing.

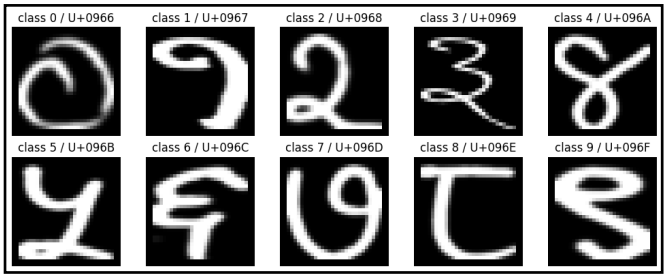
The system is trained on the digit subset of the UCI Devanagari Handwritten Character Dataset. It consists of a PyTorch training notebook for data loading, training, evaluation and model export and a Streamlit web app for interactive inference.

2. Dataset

The dataset used is the UCI Devanagari Handwritten Character Dataset, id 389. The full dataset contains 46 classes, including digits and characters. For T3.1 only the folders digit_0 through digit_9 are used. The official training and test folders are preserved and a stratified validation split is created from the official training data.

Each image is loaded as grayscale and resized to 32x32 pixels. Pixel values are converted to floating point and normalised to the range [0, 1]. The classes are balanced with 1700 training images and 300 test images per digit in the official dataset folders.

Split	Images	Use
Training	15300	Model fitting after holding out validation samples
Validation	1700	Model selection and best-checkpoint saving
Test	3000	Final evaluation on the official test split
Total digit subset	20000	10 balanced digit classes



3. Method

3.1 Model Architecture

The classifier is a compact convolutional neural network designed for small grayscale symbols. It has three convolutional blocks followed by a small fully connected head. Batch normalisation stabilises training, ReLU introduces non-linearity, max pooling reduces spatial resolution and dropout helps reduce overfitting.

Stage	Operation	Purpose
Input	1 x 32 x 32 grayscale image	Compact representation matching the dataset
Conv block 1	Conv 1->32, BatchNorm, ReLU, MaxPool	Low-level stroke and edge features
Conv block 2	Conv 32->64, BatchNorm, ReLU, MaxPool	Intermediate curves and digit parts
Conv block 3	Conv 64->128, BatchNorm, ReLU, Dropout2d	Higher-level digit structure
Classifier	Flatten, Linear 8192->24, ReLU, Dropout, Linear 24->10	Maps learned features to 10 digit classes

The resulting model has 290,002 trainable parameters which is close to the assignment's suggested 300k parameter scale. No transfer learning is used because the problem is simple enough for a task specific CNN and because

the handwritten digit domain differs from natural-image pre-trained model assumptions.

3.2 Training Procedure

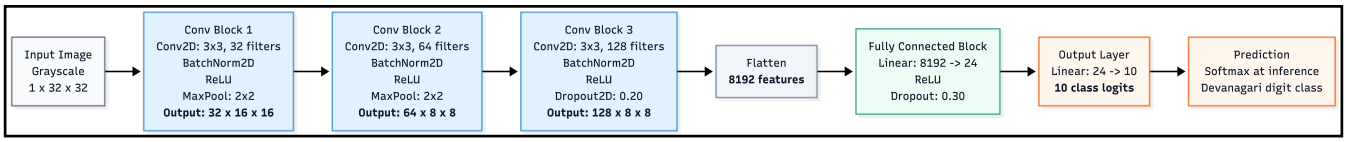
- Loss function: cross-entropy loss for 10 mutually exclusive digit classes.

- Optimizer: Adam with an initial learning rate of 1e-3.

- Scheduler: ReduceLROnPlateau to lower the learning rate when validation loss stalls.

- Epoch budget: up to 12 epochs with patience-based early stopping logic.

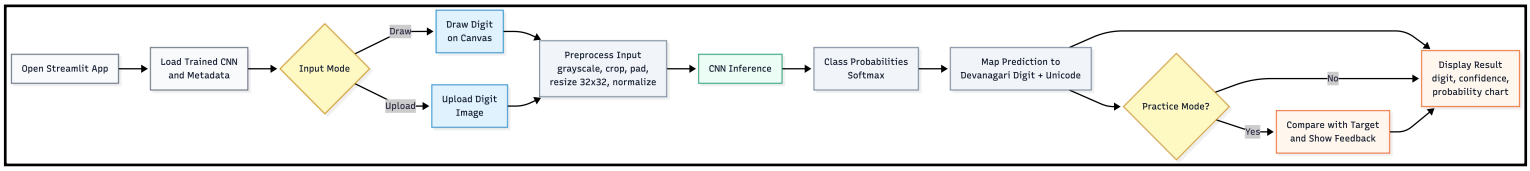
- Checkpointing: the best validation-accuracy model is saved as devanagari_digit_cnn.pt.



4. Web Application

The user-facing application is implemented in Streamlit. The app is intentionally kept separate from the training notebook so that deployment only needs the inference model, class metadata and app code. This makes the demo simpler, faster to load and easier to host on Streamlit Community Cloud.

For drawn images, the canvas uses a black background and white stroke matching the light-on-dark format expected by the preprocessing pipeline. For uploaded images the app provides polarity options so both dark-on-light and light-on-dark digit images can be handled. The preprocessing function converts the input to grayscale, optionally inverts it, crops foreground pixels, pads the crop to a square, resizes it to 32x32, normalises pixel values and sends the tensor to the model.



Component	File	Role
Training	model.ipynb	Downloads data, trains the CNN, evaluates results, and exports artefacts
App UI	streamlit_app/app.py	Canvas, upload flow, preprocessing, prediction display, and charts
Model Class	streamlit_app/model.py	CNN architecture used to reload the saved checkpoint
Checkpoint	streamlit_app/models/devanagari_digit_cnn.pt	Best trained model weights
Metadata	streamlit_app/models/class_metadata.json	Class names, Unicode mapping, image size, and metrics

5. Results

The model trains quickly and achieves high accuracy on the official test split. The notebook records the training curve, validation performance, classification report, confusion matrix and misclassified examples. The final checkpoint included with the app corresponds to the best validation model.

Metric	Value	Interpretation
Best validation accuracy	99.7059%	Used for best-checkpoint selection
Official test accuracy	99.6000%	Final held-out performance
Incorrect test predictions	12 / 3000	Small number of errors across all digits
Trainable parameters	290002	Small CNN scale suitable for the assignment

The per-class report shows strong precision and recall across all ten digits. Digits 0 and 7 reached perfect precision and recall in the recorded test run, while the lowest F1-score was still approximately 0.9867 for digit 2. This suggests that the model has learned robust stroke patterns rather than relying on a single class imbalance or shortcut.

6. Error Analysis

The most frequent error was class 2 being predicted as class 3. In the misclassified examples, several handwritten 2 samples have a rounded upper

loop and descending right-side stroke making them visually similar to handwritten Devanagari 3. The lower stroke of 2 is also sometimes faint, broken or compressed after resizing to 32x32, reducing the features that distinguish it from 3.

This confusion is likely caused by natural handwriting variation rather than a systematic model failure. Since the CNN learns pixel-level stroke patterns-ambiguous 2-like shapes with weak lower structure can shift the prediction toward class 3.

7. Limitations

- The model is trained on isolated 32x32 digit images so performance may drop on noisy camera photos, touching digits or full document scans.
- The app predicts one digit at a time and does not perform segmentation for multi-digit numbers.
- Canvas handwriting style may differ from the dataset distribution, especially for mouse-drawn inputs.
- The preprocessing is rule-based and may fail if the foreground/background contrast is very weak.
- The current model only supports Devanagari digits and does not recognise the full 46-class Devanagari character set.

8. Conclusion

This project successfully implements a handwritten Devanagari digit recogniser for the T3.1 assignment variant. A small PyTorch CNN was trained from scratch on the digit subset of the UCI Devanagari Handwritten Character Dataset. The model achieved 99.6% accuracy on the official test set, exceeding the assignment target of 97%+. A Streamlit app was built with drawable canvas input, upload input, confidence visualisation, unicode-aware output and practice mode.

The final system demonstrates that a simple well-designed CNN can solve isolated handwritten Devanagari digit recognition effectively while remaining lightweight enough for a realtime web demo.

9. LLM Usage Acknowledgement

LLM tools (ChatGPT) were used for assistance with code scaffolding, debugging and documentation phrasing/structure.

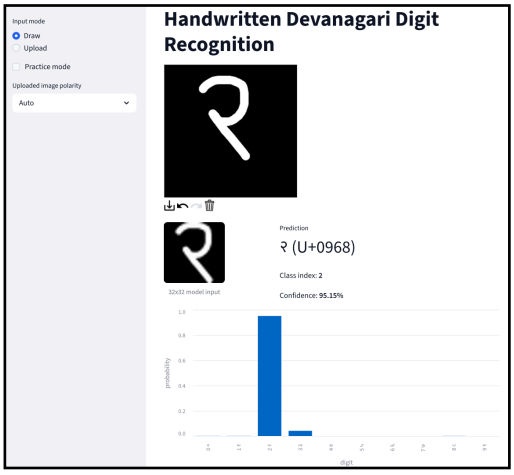
10. References

1. Acharya, S. & Gyawali, P. (2015). Devanagari Handwritten Character Dataset [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XS53.

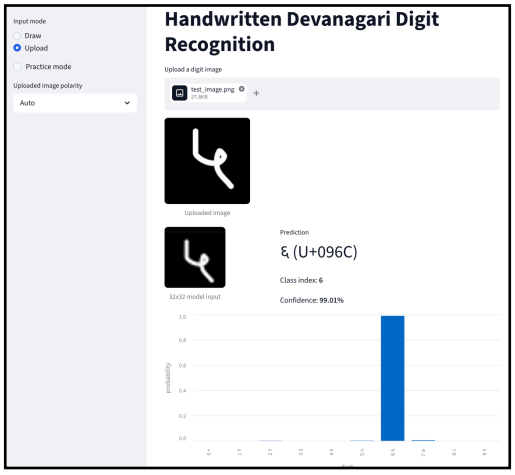
2. Prashanth, Duddela & Mehta, R Vasanth Kumar & Sharma, Nisha. (2020). Classification of Handwritten Devanagari Number – An analysis of Pattern Recognition Tool using Neural Network and CNN. Procedia Computer Science. 167. 2445-2457. 10.1016/j.procs.2020.03.297.

3. Streamlit Documentation - https://docs.streamlit.io/

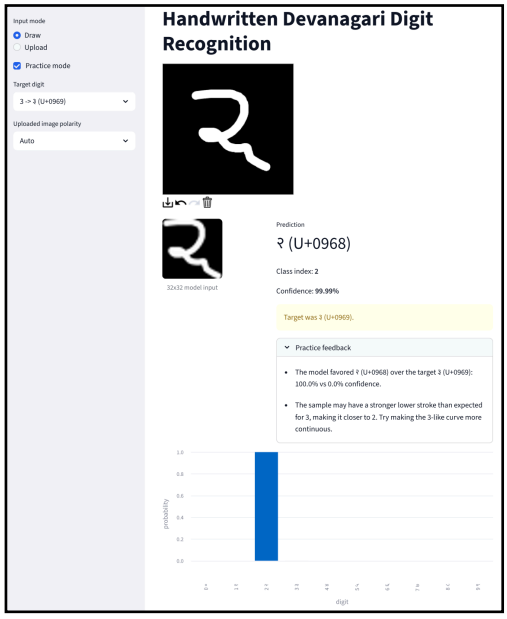
11. App Images (Taken from deployed streamlit app)



Draw Input Mode



Upload Input Mode



Practice Mode with target class choices and Live Feedback